CONTROLLER ROUTER

Integration Guide

Build a unique controller for your game,
using shared routing and display APIs.

VIRTUALGLOVE / R.O.B. VISION / MAKER CONTROLLERS

Current source edition | 27 October 2026

Controller Router is a reusable Linux input-routing library for unusual controllers and games. Your project supplies the controller, game-specific interpretation, and receiver. Router combines that input with ordinary gamepads and exposes merged Players 1 through 4 to RetroArch.

The optional UNO Q runtime adds shared app selection, an input lease, and a 13 by 8 Matrix API. You can use the console library without the UNO Q runtime.

Choose an integration path

Your controller	Recommended boundary
A device already configured in EmulationStation	Import its mapping and assign its discovered source ID
A custom receiver that creates a Linux uinput gamepad	Describe its fixed controls in a source descriptor, then assign it
Gesture state similar to VirtualGlove	Use the existing local datagram contract, reviewing its game/core gating
Native game-specific data outside RetroPad	Keep that transport in your project; use Router for companion physical pads

For a new controller, a fixed uinput device and source descriptor provide the clearest starting point. The datagram path currently carries VirtualGlove-shaped state and enables virtual input for recognised joystick cores. It is not an unrestricted controller protocol for every game.

Define the project boundary

1. Choose the game and required controls: buttons, directions, analogue values, or native data.
2. Implement the sensor or device reader in your project.
3. Translate its readings into the game's controls in your adapter.
4. On the console, create a stable uinput gamepad identity or an explicit native transport.
5. Give Router the input mappings and chosen player assignment.
6. Authenticate remote commands and release held controls if communication stops.
7. Add a setup interface that calls Router's assignment API.
8. Test the game's actual input, exit behaviour, and disconnect recovery.

Router does not detect ROMs, know game rules, interpret a sensor, define your product-specific game credentials, or provide a generic game plugin loader. Those responsibilities stay with the host project.

Describe a custom input source

A fixed source descriptor identifies a controller by name, vendor, and product. Its mapping connects familiar logical controls to Linux event codes. It does not choose the player.

```
{
  "schema": 1,
  "sources": [{
    "name": "My Arcade Lever",
    "vendor": "1209",
    "product": "0002",
    "mapping": [{
      "name": "a", "type": "button", "code": 0,
      "value": 1, "evdev_code": 304
    }]
  }]
}
```

This single-button example is a descriptor, not a complete controller driver. Your receiver must create the matching device and emit its events. Choose identifiers appropriate to your project; do not copy a physical product's identity.

Set `CONTROLLER_ROUTER_SOURCES_FILE` in the Router service environment to your descriptor's path. Read Router inventory to obtain the source ID, assign it to a player, and save using the current revision. Keep the descriptor installed with your receiver so the engine can rediscover it after restart.

Build the assignment interface

The shared `RouterStore` API supplies inventory, configuration, test activity, save, and rollback. Your project can expose those operations through its authenticated setup service.

- **Read:** display available sources, their current assignments, and the returned revision.
- **Test:** call `check` with a bounded `watch_ms` interval and show detected input activity.
- **Save:** send source IDs grouped by player, together with the revision read by the page.
- **Restore:** request `rollback` with the current revision when a previous snapshot exists.

A source can contribute to a merged player alongside a physical controller. The engine supports Players 1 through 4. It keeps compatibility output names such as `VirtualGlove Merged Player 2`; that name does not require VirtualGlove to be installed.

Finish a game before saving or restoring assignments. Router rejects relevant active-game reconfiguration and stale revisions. If another page changed settings, reload the state rather than forcing a save.

Example: R.O.B. Vision

R.O.B. Vision receives commands from registered NES games through its emulator integration. Its adapter turns those commands into Buddy's virtual actions and Gyromite gate button state. Its console receiver creates a fixed controller source; Router merges that source into Player 2.

R.O.B. Vision keeps its rule that Buddy must stay on Player 2 in its own authenticated `/router` adapter. It calls the generic `RouterStore.operate()` API after checking that rule. The library does not hard-code Buddy's game rules.

On the UNO Q, the app asks the shared display client to play animations under `rob_vision`. Its game session selects the app and grants its input lease. The visual robot and gyro simulation remain R.O.B. Vision code.

Sources: R.O.B. Vision Router setup, authenticated assignment adapter, and Matrix adapter.

Example: VirtualGlove

VirtualGlove interprets camera gestures, selects a game profile, and sends gesture state through its receiver. Router combines the permitted virtual state with the assigned physical inputs. Native Super Glove Ball hand data uses a separate product transport; Router still supplies physical gamepad and hotkey input.

VirtualGlove wraps `RouterStore` in `RouterService`. It authenticates configuration operations with a signed, short-lived challenge before calling the shared API. That transport belongs to VirtualGlove; a new project can choose its own authenticated transport without changing the storage engine.

Its UNO Q Matrix adapter calls the same display API with the app ID `virtualglove`. The shared firmware renders the selected app's request rather than loading another product sketch.

Sources: VirtualGlove authenticated Router facade and Matrix adapter.

Add an optional UNO Q display

Supply a schema-1 animation manifest and call the local display API to request named animations, short status text, or temporary pairing codes. Keep animation design and game-specific cue choices in your app.

The current portal explicitly allows VirtualGlove and R.O.B. Vision. A third controller needs registration in its allowed-app lists, health endpoints, installer, and manifest paths before it can call the display API. Adding a manifest alone does not register an app. The Technical Reference describes this extension boundary.

Router grants one active app lease. Your app and receiver must release input when that lease is missing, expired, revoked, or belongs to a previous boot. A website should remain optional: your authenticated live-game session should drive automatic selection.

Validate your controller

1. Confirm your receiver's device identity and event codes match its descriptor.
2. Read inventory, save an assignment, restart, and confirm it persists.
3. Test each control independently in the real game.
4. Confirm physical and custom inputs reach the intended player and preserve hotkeys.
5. Disconnect the custom controller while a button is held; verify release.
6. Try a stale revision and an assignment save during a game; verify rejection.
7. Test a software upgrade without losing assignments or pairing credentials.
8. If using UNO Q ownership, test selection, lease expiry, game exit, and service failure.

Keep merged outputs alive while physical pads sleep, wake, or reconnect. The shared launch adapter resolves their identities immediately before execution and supplies temporary legacy indexes or strict native reservations. A source reconnect must not recreate the merged outputs. If Router itself loses those outputs, require the player to end the game and relaunch after recovery. Test real wireless hardware as well as the automated reconnect fixture; see Routing validation.

Expose system choices

Save system policy with player assignments through the same revision-checked configuration API. `physical_scope` accepts `nes`, `all`, or `systems`; selected mode uses `physical_systems` for canonical system IDs. Preserve the current policy when an older client updates only player assignments.

A disabled system keeps its original launch arguments and receives no Router routing overrides or physical-source forwarding. Keep merged outputs connected. The Technical Reference defines validation and launch behaviour; the User Guide provides the player-facing Setup instructions.

Use the shared connection authority

Declare your installed console adapter with fixed credential and destination paths plus fixed restart and stop commands. Register its UNO adapter with a capability file. Reuse Router’s secure Pair console page instead of collecting SSH passwords. VirtualGlove imports its signed profile/input credential; R.O.B. Vision imports a console-scoped bearer credential and adds Buddy to Player 2. Keep game registries and actions in your product adapter.

Native RetroPie host adapters

The shared build includes `retropie_native`, `native_merged_input`, and native-session discovery. The installer stages `/opt/controller-router/bin/native-route` and covers direct Libretro commands in top-level and nested port registrations. It does not replace custom launch-script arguments or automatically convert native emulator settings.

Native behavior stays in the host adapter. Install `/etc/controller-router/native-adapters.json` (schema 1) with explicit provider paths and exact command SHA-256 registrations keyed by `emulators.cfg` path. A provider exposes `native_session(proc_root)` returning `(adapter_kind, system)` or `(None, None)`. Use the same detection for the daemon and Setup so saves/rollback are blocked during native play. Treat provider files and this manifest as administrator-managed executable configuration; never accept provider paths from web requests.

The native registry retains the original command templates and defines runtime `user`, `state_root`, and `launcher`. Preserve console settings, pointers/lightguns, credentials, ROM registrations, existing qualified launchers and fallbacks. Disabled routing passes the original command through without merged-device setup. New native installations require explicit host qualification; catalogue presence means an installed adapter, not a blanket game certification.

Configuration recovery must contain text configuration and custom routing source only. Keep emulator executables, Libretro cores, shared libraries and build output out of `backup.sh` archives and `update.sh` restores. Record their paths, versions and checksums as separately installed prerequisites.